

BUT 2 INFORMATIQUE
ANNÉE UNIVERSITAIRE 2025-2026

Rapport de Stage

Database Migration for MORFETIK

Stage effectué du 02/02/2026 au 27/03/2026

Auteur : Saïf-eddine ALJANE

Enseignant référent : Jean-Christophe DUBACQ

Tuteur de stage : Jaime ARIAS

Établissement : IUT de Villetaneuse (Université Sorbonne Paris Nord)

Organisme d'accueil : Laboratoire d'Informatique de Paris Nord (LIPN)
Institut Galilée, 99 Av. Jean Baptiste Clément, 93430 Villetaneuse

Remerciements

Avant d’entamer ce rapport, je tiens à remercier toutes les personnes qui ont contribué au bon déroulement de mon stage.

Tout d’abord, je tiens à remercier mon maître de stage, **Jaime ARIAS**, ingénieur de recherche et responsable de la dev-team (équipe de développement logiciel du **LIPN**), pour le temps qu’il a consacré à m’aider et à répondre à mes questions, ainsi que pour ses nombreux conseils qui m’ont permis de progresser.

Je tiens aussi à remercier l’ensemble de la dev-team, et notamment **Othman BOU-DARGA** (Apprenti ingénieur) ainsi qu’**Aude GREZKA** (Ingénieure de recherche et linguiste). Étant donné que ce sont eux qui ont la charge du projet MORFETIK aux côtés de **M. ARIAS**, ils m’ont aidé à mieux comprendre le projet et ont pris le temps de répondre à toutes mes questions.

Enfin, je tiens à remercier **M. MARTINEZ** pour son aide précieuse lors de mes recherches de stage, ainsi que **M. DUBACQ** pour avoir assuré mon suivi tout au long de cette expérience.

Table des matières

1	Introduction	6
2	Le laboratoire	8
2.1	L'Université Sorbonne Paris Nord (USPN)	8
2.2	Le Centre National de la Recherche Scientifique (CNRS)	8
2.3	Le Laboratoire d'Informatique de Paris Nord (LIPN)	9
3	Environnement professionnel et cadrage du projet	12
3.1	Le cadre professionnel	12
3.2	Vue d'ensemble de MORFETIK	13
3.2.1	Présentation de l'outil MORFETIK	13
3.2.2	Contexte de la refonte de Morfetik	14
3.2.3	Justification de la migration vers PostgreSQL	14
3.3	Architecture et outils de développement	14
3.3.1	Architecture logicielle de MORFETIK	14
3.3.2	Technologie utilisée	15
3.4	Méthodologie de travail	17
4	Analyse et réalisations techniques	20
4.1	Analyse des données de Morfetik	20
4.2	Conception et développement de la solution de migration	23
4.3	Analyse des problématiques techniques et solutions apportées	25
4.3.1	Gestion des clés étrangère	25
4.3.2	Les documents imbriqués	26
4.3.3	Validation et transformation des types	27
5	Conclusion et perspectives	28
	Bibliographie	30
	Glossaire	31
	Annexes	34
1	Statistiques de la base de données MORFETIK	35
2	Préfixes pour les commits	35
3	Schéma relationnel de la base de données PostgreSQL (ERD)	36
4	Modèle de données de la collection « Nouns »	37
5	Exemple d'un document de la collection « Nouns »	37
6	Exemple d'un script de migration de données	38

7	Script d'orchestration	39
8	Dictionnaire des chemins de conjugaison (conjugation_path)	40
9	Fonctions utilitaires (tools.py)	40
10	Arborescence du système de migration	41

Table des figures

2.1	Organigramme détaillé du LIPN	10
3.1	Aperçu de mon poste de travail au laboratoire LIPN	12
3.2	Prix EGC récompensant le projet MORFETIK	13
3.3	Exemple d'un document extrait de la collection "nouns"	15
3.4	Schema de la gestion des versions avec Gitflow	18
3.5	Revue de code lors d'une demande de fusion (merge request)	19
4.1	Définition et paramétrage des contraintes d'intégrité de la table nouns_codes	21
4.2	Dictionnaire des domaines	21
4.3	Processus de transformation et d'insertion des données	24
4.4	Création de la liste de tuples de correspondance Code/UUIDv7.	26

CHAPITRE 1

Introduction

Dans le cadre de ma deuxième année de BUT informatique au sein de l'IUT de Villeneuve, j'ai intégré le **Laboratoire d'Informatique de Paris Nord (LIPN)** pour une durée de 2 mois. Cette opportunité m'a permis d'avoir une nouvelle perspective sur l'apprentissage en m'invitant à adapter mes compétences théoriques et pratiques acquises lors de mon cursus aux réalités du monde professionnel. Elle m'a aussi apporté une solide expérience professionnelle qui me sera d'une grande utilité pour la suite de mon cursus et mon avenir en entreprise.

Au sein du **LIPN**, j'étais rattaché à l'équipe de développement logiciel du laboratoire (dev team). Cette équipe est dirigée par Jaime ARIAS (mon tuteur de stage). Le **LIPN** est divisé en plusieurs équipes de recherche, chacune travaille sur des thématiques différentes, mais elle reste toujours en rapport avec le domaine de l'informatique.

Le choix du **LIPN** pour mon stage se base sur le fait que les missions proposées sont entièrement en rapport avec ma formation et qu'il y a un encadrement sérieux. Ces critères sont importants, car ils garantissent que le stage soit pertinent pour mon parcours professionnel et pédagogique. De plus, lors de mes recherches de stage j'ai constaté qu'il y a peu d'offres qui proposent un réel projet de développement du fait que le stage ait une durée de 2 mois, ce qui est très court. Donc j'ai privilégié un stage au **LIPN** pour pouvoir travailler sur un projet réel et être bien encadré pour pouvoir progresser.

Lors de mon stage ma principale mission était de migrer des données d'une base de données NoSQL (MongoDB) vers une base de données relationnelle (PostgreSQL). Cette base de données appartient à MORFETIK. MORFETIK est une ressource lexicale pour le traitement automatique du langage. Cette ressource se matérialise sous forme d'une application web. Ma mission s'inscrit dans un projet plus vaste, qui est de faire migrer l'ancienne version de MORFETIK vers une version plus moderne. Ainsi, mon objectif était de créer une stratégie de migration des données qui garantit la cohérence et l'intégrité des données, malgré la différence de paradigme entre les bases de données NoSQL et les bases de données relationnelles.

Afin de communiquer mon expérience, ce rapport va s'articuler autour de trois parties. Tout d'abord, je présenterai l'organisation du **LIPN**. Ensuite, je parlerai plus en détail

de ce qu'est MORFETIK et des technologies que j'utilise, tout en précisant la méthode de travail adoptée. Enfin, je détaillerai la mise en œuvre de mes missions, en mettant en avant les solutions que j'ai trouvées pour accomplir mes objectifs.

CHAPITRE 2

Le laboratoire

J'ai été accueilli par le **Laboratoire d'Informatique de Paris Nord (LIPN)** pour mon stage. Le **LIPN** est une unité mixte de recherche (UMR 7030). Entre le **Centre National de la Recherche Scientifique (CNRS)** et l'**Université Sorbonne Paris Nord (USPN)**.

2.1 L'Université Sorbonne Paris Nord (USPN)

L'**Université Sorbonne Paris Nord (USPN)** est une université publique implantée sur plusieurs campus en Seine-Saint-Denis et dans le Val-d'Oise, tous situés au nord de Paris.

Elle a été fondée en 1970 après la scission de l'**Université de Paris** en treize universités distinctes. C'est de là que lui vient son ancien nom, **Université Paris 13**. En 2020, elle change de nom pour "**Université Sorbonne Paris Nord**" afin de garder son identité géographique et son appartenance à la Sorbonne. Elle propose des enseignements dans une multitude de domaines tels que les sciences, la santé, le droit, l'économie, la gestion, les lettres, les langues, ainsi que les sciences humaines et sociales.

Pour finir, l'**USPN** est reconnue dans le domaine de la recherche scientifique grâce aux nombreux laboratoires de recherche sous sa tutelle, dont le **LIPN** fait partie.

2.2 Le Centre National de la Recherche Scientifique (CNRS)

Le **Centre National de la Recherche Scientifique (CNRS)** est un organisme public français consacré à la recherche scientifique. C'est l'organisme le plus reconnu en France en termes de recherche dans tous les domaines scientifiques, il bénéficie aussi d'une renommée mondiale. Le **CNRS** dépend du **Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation (MESRI)**.

En 2025, le **CNRS** compte plus de **1 100 laboratoires** et compte près de **34 700 membres**, parmi lesquels **29 400 scientifiques**. Le **LIPN** est l'un de ces laboratoires.

De plus, si certaines unités sont liées à des universités (comme c'est le cas pour le **LIPN** avec l'**USPN**), d'autres collaborent avec des industries, des **Établissements Publics à caractère Scientifique et Technologique (EPST)** ou de grandes écoles.

Ainsi en tant qu'organisme pluridisciplinaire, le **CNRS** est chargé de nombreuses missions. Sa mission principale est de promouvoir et partager les avancées scientifiques issues de ses laboratoires.

2.3 Le Laboratoire d'Informatique de Paris Nord (LIPN)

Le **Laboratoire d'Informatique de Paris Nord (LIPN)** a été fondé en 1985 , mais il est rattaché au **CNRS** seulement depuis 1992 . Il fait aussi partie de la fédération de recherche **MathSTIC** (FR 3732), qui fédère les trois laboratoires de recherche de **USPN** (le **LIPN**, le **LAGA** et le **L2TI**). Cet ensemble de laboratoires a pour domaine de recherche les mathématiques, les sciences et technologies de l'information et de la communication.

Plus précisément, le **LIPN** est composé de 6 équipes de recherche, des membres permanents (chercheurs **CNRS**, professeurs et maîtres de conférences de **USPN** ou ingénieurs du **CNRS**). Ils sont environ 70. Il y a le personnel technique et administratif, ils sont 10. Et il y a aussi des doctorants et des post-doctorants. Globalement, il y a **150 membres** sans prendre en compte les stagiaires et les visiteurs. (voir **figure 2.1**).

Les recherches du **LIPN** s'articulent principalement autour de ces domaines : combinatoire, optimisation combinatoire, algorithmique, logique, génie logiciel, systèmes distribués, langage naturel et apprentissage. Ces domaines de recherche sont répartis entre les équipes du laboratoire, chacune se consacrant à un ou plusieurs de ces domaines.

Voici les noms de ces 6 équipes :

- **A3** : Apprentissage Artificiel et Applications,
- **AOC** : Algorithmes et Optimisation Combinatoire,
- **CALIN** : Combinatoire, Algorithmique et Interactions,
- **LoCal** : Logique et Calcul,
- **RCLN** : Réprésentation des Connaissances et Langage Naturel,
- **SAFER** : Safety and Security Analys^{is} via Formal and Efficient veRification.

Ainsi chaque équipe travaille sur un domaine précis ce qui donne au **LIPN** le statut de laboratoire pluridisciplinaire dans le domaine de l'informatique.

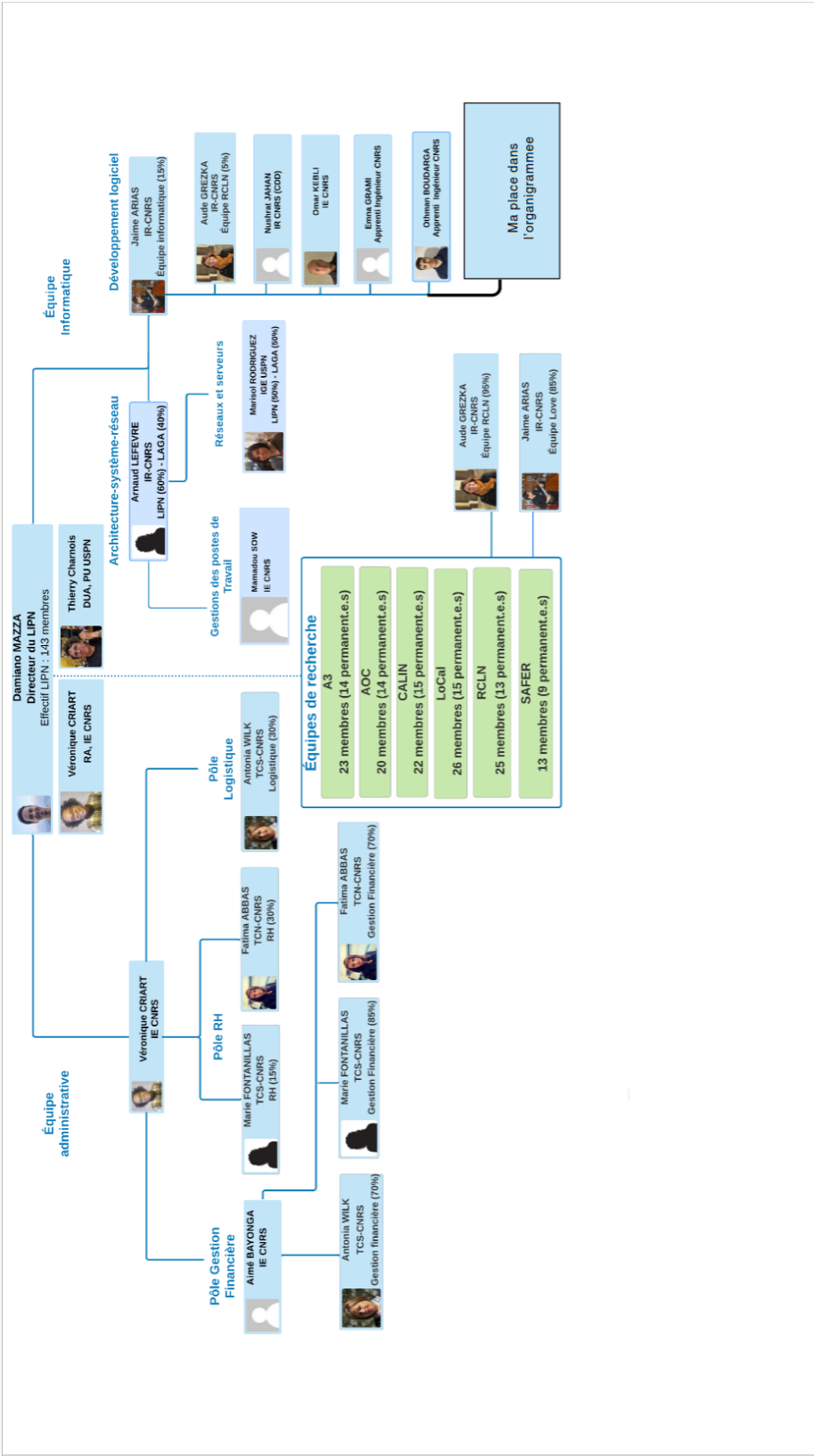


FIGURE 2.1 – Organigramme détaillé du LIPN

L'équipe de développement logiciel (dev-team)

La **dev-team** est l'équipe de développement logiciel du **LIPN**. Elle a pour mission de concevoir des solutions logicielles qui profitent au laboratoire. Cette mission exige une approche multidisciplinaire afin d'avoir le plus de points de vue possibles et ainsi pouvoir développer efficacement n'importe quels logiciels qui serviraient au laboratoire.

Parmi les réalisations de la dev-team figurent le projet **Morfetik**, sur lequel j'ai travaillé durant mon stage. L'équipe travaille aussi sur d'autres projets tels que la refonte de l'intranet du **LIPN**.

Cette équipe est dirigée par **Jaime ARIAS** (mon tuteur de stage). Elle se compose de trois ingénieurs de recherche **CNRS** (dont un en disponibilité) et de deux apprentis issus de l'école d'ingénieurs **SupGalilée**. Un ingénieur d'études **CNRS**, actuellement en disponibilité, complète l'équipe. Chaque année, la dev-team accueille également des ingénieurs en CDD et des stagiaires. Lors de mon stage, nous étions six étudiants en BUT 2, chacun et chacune affectés à des missions distinctes sur des projets variés.

Environnement professionnel et cadrage du projet

3.1 Le cadre professionnel

Le début de ma période de stage a été marqué par une situation assez particulière. En effet, cette période coïncidait avec le **déménagement** du laboratoire dans ses nouveaux locaux. Ainsi, durant les **quatre premières semaines**, j'ai partagé un bureau avec un autre stagiaire et deux chercheurs. À la suite du déménagement, j'ai été installé dans **l'espace exclusivement dédié à la dev-team**, où tous les **stagiaires de BUT 2** étaient réunis. Cela a favorisé la **communication** entre nous, nous pouvions nous **entraider** lorsque l'un de nous était en difficulté.



FIGURE 3.1 – Aperçu de mon poste de travail au laboratoire LIPN

Concernant mon **équipement**, j'ai reçu un **ordinateur portable vierge** sans système d'exploitation, j'ai donc dû en **installer un moi-même**. J'ai choisi **Xubuntu** comme système d'exploitation, car c'est la distribution GNU/Linux avec laquelle je suis le plus fa-

milier.

Pour ce qui est de l'**encadrement professionnel**, il est assuré par mon tuteur de stage. À chaque fois que je développe une fonctionnalité ou tout autre livrable, il s'assure que mon travail respecte certains critères. Ces critères de qualité portent sur la propreté du code, la qualité de la documentation et le respect des conventions de « workflow » de la **dev-team** (cf. [section 3.4](#)). Ces critères ont pour but de faciliter la reprise de mon travail par quelqu'un d'autre, même longtemps après mon départ. Ils permettent aussi de garantir un standard de qualité élevé, très important au sein d'un laboratoire comme le **LIPN**. Ces exigences font écho aux bonnes pratiques de développement acquises durant ma formation en **BUT**.

3.2 Vue d'ensemble de MORFETIK

3.2.1 Présentation de l'outil MORFETIK

MORFETIK est une ressource lexicale dédiée au **Traitement Automatique du Langage (TAL)**. Plus précisément, il s'agit d'un **dictionnaire morphologique** muni d'un moteur de flexion. Ce type d'outil permet de référencer les formes de base des mots (*i.e.* **lemmes**) ainsi que l'intégralité de leurs variantes (*i.e.* **flexions**). Contrairement aux dictionnaires classiques qui ne répertorient que les lemmes pour en donner la définition, MORFETIK permet de générer toutes les flexions d'un mot à partir de son lemme, ou inversement, d'identifier un lemme à partir d'une de ses formes fléchies. L'outil fournit également des informations sémantiques et recense actuellement plus de **1 000 000** de formes (cf. [Annexe 1](#)).

La pertinence de MORFETIK a été reconnue, le projet ayant obtenu un prix lors de la **26ème Conférence Francophone sur l'Extraction et la Gestion des Connaissances (EGC 2026)** :



FIGURE 3.2 – Prix EGC récompensant le projet MORFETIK

3.2.2 Contexte de la refonte de Morfetik

Maintenant que nous savons ce qu'est MORFETIK, je peux expliquer en détail quelle a été ma mission durant ce stage. MORFETIK est une ressource existant depuis une vingtaine d'années. Elle a été reprise en 2015 par **Aude GREZKA** (Ingénieure de Recherche CNRS et linguiste) afin de la faire évoluer. C'est dans cette dynamique d'évolution qu'une refonte complète de l'architecture technique de MORFETIK a été décidée en amont de mon stage.

L'ancienne version reposait sur la pile technologique **MEVN** (*i.e.* MongoDB, Express, Vue.js, Node.js). La nouvelle version utilise désormais **Adonis JS**, **POSTGRES SQL** et **Vue JS**. Parallèlement, l'architecture logicielle a évolué vers une **architecture hexagonale** (*cf.* [section 3.3](#)). Ce choix de conception permet de séparer strictement la logique métier des détails techniques d'infrastructure et ainsi réduire la dette technique.

3.2.3 Justification de la migration vers PostgreSQL

Les données utilisées par MORFETIK sont profondément liées entre elles. Un **lemme** est lié à toutes ses **flexions**, mais aussi à une multitude d'informations sur sa sémantique. Il est donc impératif de garder ces relations dans notre base de données. C'est là que MONGODB a des lacunes, car en tant que base de données orientée document (**NoSQL**) elle ne permet pas de mettre en relation les données de façon native. Il faut ajouter une logique au niveau du code pour reproduire le fonctionnement d'une base de données relationnelle afin de mettre en relation les données. Il est donc préférable de migrer vers **POSTGRES SQL**, une véritable base de données relationnelle, plutôt que de chercher à simuler cette spécificité au niveau du code. C'est pour ces raisons qu'il m'a été demandé de développer une solution afin de migrer les données de la version MONGODB vers la version **POSTGRES SQL** (*cf.* [chapitre 4](#)).

3.3 Architecture et outils de développement

3.3.1 Architecture logicielle de MORFETIK

La nouvelle version de MORFETIK adopte une **architecture hexagonale**. Bien que cette architecture ait été implémentée avant le début de mon stage, j'ai dû en comprendre les bases pour bien appréhender le contexte du projet. L'architecture hexagonale permet de séparer la logique métier, ici la gestion des **lemmes** et de leurs **flexions** des détails techniques. Au cœur de cette architecture se trouve le **domaine**, c'est là où se trouvent les règles de **la logique métier**. Ce qui représente les côtés de l'hexagone sont les **ports**, il s'agit d'**interfaces** qui définissent comment interagir avec le domaine. Chaque port est rattaché à une **dépendance extérieure** (*e.g.* base de données, interface utilisateur, services tiers) via un **adaptateur**. Cette manière de faire permet d'atténuer **la dette technique** et d'éviter un **couplage fort** entre le cœur du système et les outils extérieurs. Cela est rendu possible par l'isolation du domaine du reste du système. Ainsi, si l'une des dépendances extérieures ne convient plus, il suffirait de développer un autre adaptateur sans rien modifier d'autre, cela facilite grandement la maintenabilité du code.

3.3.2 Technologie utilisée

Afin de mener à bien ma mission, j'ai utilisé un ensemble de technologies et de langages de programmation. J'ai découvert de nouvelles technologies, mais j'ai également utilisé des outils que je connaissais déjà.

MongoDB

L'ancienne version de MORFETIK utilise MONGODB. MONGODB est un **Système de Gestion de Base de Données (SGBD)** de type **NoSQL** orienté document. Les données de ce type de base de données sont stockées sous la forme de fichiers **BSON** (très similaires au **JSON**).

A screenshot of a MongoDB document in BSON format, displayed in a dark-themed code editor. The document is a JSON-like object with the following fields: `_id` (ObjectId), `name` (string), `catgram` (string), `souscat` (string), `flexion` (string), `variante` (string), `infos` (string), and `notes` (null).

```
{
  "_id": ObjectId('64e7798673cae392d83a65ec'),
  "name": 'agissant',
  "catgram": 'adj',
  "souscat": '',
  "flexion": '42',
  "variante": '',
  "infos": '',
  "notes": null
}
```

FIGURE 3.3 – Exemple d'un document extrait de la collection "nouns"

Chaque document est en réalité un objet qui possède des attributs sous format **clé-valeur**. Ces documents sont regroupés au sein de **collections**, et une base de données est composée d'une ou plusieurs collections.

Ce qui distingue MONGODB des **SGBD** qui ne sont pas orientés document est la **flexibilité**. En effet, les documents sont "**schema-less**", ce qui signifie qu'au sein d'une même collection, deux documents peuvent avoir des attributs différents sans que le **SGBD** ne relève d'erreur. De plus le type de données est, lui aussi, flexible.

Cette flexibilité est précisément ce qui rend la migration vers PostgreSQL complexe. Contrairement à MONGODB, PostgreSQL impose un **schéma strict** qui ne laisse aucune place aux données mal structurées ou incomplètes.

PostgreSQL

Comme dit plus tôt, PostgreSQL est très différent de MONGODB. PostgreSQL est un **Système de Gestion de Base de Données Relationnel (SGBDR)**, c'est-à-dire qu'il stocke et agence les données par rapport aux relations qu'elles ont entre elles. Ici, les données sont stockées sous forme de lignes dans des **tables**. Chaque colonne indique à quoi correspond la donnée. Dans ce type de base de données, les tables sont créées à partir d'un **schéma strict** auquel on ajoute des **contraintes** sur le type de la donnée.

Cela implique qu'il n'y a pas de place pour les données incomplètes ou ne suivant pas le schéma. Cela rend les bases PostgreSQL très robustes.

Cette robustesse est garantie par le respect des propriétés **Atomicité, Cohérence, Isolation, Durabilité (ACID)** :

- **L'atomicité** permet de garantir qu'une transaction (requête qui interagit avec la base de données) soit effectuée entièrement ou pas du tout. Cela implique que si, lors de la transaction, un problème technique survient et l'interrompt en plein milieu, le **SGBD** ne prend pas en compte les modifications partielles et ramène la base à son état initial.
- **La cohérence** permet d'assurer que chaque modification de la base de données respecte les contraintes définies par les schémas. Cela garantit que seules des données valides (respectant les types et les règles établies) peuvent être enregistrées.
- **L'isolation** permet d'éviter les conflits lorsque plusieurs utilisateurs veulent interagir avec les données.
- **La durabilité** qui garantit que les données soient sauvegardées de façon permanente même en cas de défaillance technique

Le fait que PostgreSQL respecte les propriétés **ACID** constitue un atout majeur pour la précision des données de MORFETIK.

Python

J'ai utilisé **PYTHON** car c'est le langage de référence lorsque l'on veut manipuler des données. Même si ma mission aurait pu être réalisée en **JavaScript**, j'ai privilégié PYTHON car c'est le langage avec lequel je suis le plus familier. Pour manipuler au mieux les collections et les documents de MONGODB, j'ai utilisé PYMONGO, une bibliothèque PYTHON qui permet de manipuler ce format de données. PYTHON permet de manipuler les collections MONGODB comme des listes de dictionnaires, et les documents comme des **dictionnaires**.

Cela est très puissant car ça permet d'utiliser les outils natifs de PYTHON (*e.g.* boucle, propriétés des dictionnaires, syntaxe familière de PYTHON) avec les données provenant de MONGODB. J'ai aussi utilisé toutes les bibliothèques et outils natifs de PYTHON pour le scripting tels que subprocess, os et json. Ainsi PYTHON est pour moi le langage le plus adapté au contexte de ma mission.

AdonisJS

Pour le **back-end** de MORFETIK, il a été décidé d'utiliser **AdonisJS**. Il s'agit d'un framework **Node.js** complet qui se distingue par l'intégration native d'une multitude de fonctionnalités telles que l'authentification ou un **Object-Relational Mapping (ORM)**. Cela permet d'avoir un code bien organisé, optimisé et maintenable. De plus, **AdonisJS** est parfaitement compatible avec **l'architecture hexagonale** du projet.

Enfin, l'un des plus grands avantages de ce framework est qu'il impose l'utilisation de **TypeScript**. **TypeScript** est une surcouche de **JavaScript** qui introduit un **typage statique**. Cela signifie que le type de chaque variable est défini explicitement dans le code, ce

qui permet de détecter les erreurs dès l'écriture, avant même l'exécution du programme.

Vue.js

Pour le front-end de MORFETIK, il a été décidé d'utiliser **Vue.js**. C'est un framework **JavaScript** qui permet de construire des pages dynamiques et réactives. **Vue.js** permet de créer une **Single Page Application (SPA)**. Avec une **SPA**, nous n'avons pas à recharger la page à chaque clic, ce qui rend l'expérience utilisateur plus fluide et confortable. **Vue.js** est aussi pratique dans notre contexte car, du fait qu'il soit un framework **JavaScript** tout comme **AdonisJS**, la liaison entre les deux se fait de manière naturelle.

Docker

Pour garantir que l'environnement de développement soit identique pour tous les membres de la *dev-team*, j'ai utilisé **docker**.

L'un des avantages majeurs de cet outil est qu'il m'a permis de faire fonctionner l'intégralité de MORFETIK sur mon poste sans avoir eu besoin d'installer manuellement tous les services et dépendances (tels que nodejs, MONGODB ou POSTGRESQL). Tout l'environnement est isolé dans des conteneurs pré-configurés, ce qui évite les erreurs de configuration et les problèmes de compatibilité entre les machines. Cela rend la mise en place du projet extrêmement rapide et fiable.

Outils de développement

Mon **IDE** principal est **VScode**. C'est l'**IDE** avec lequel je suis le plus familier.

Pour la gestion de versions, j'ai utilisé **Git** et **GitLab**. Ces outils me permettent de sauvegarder l'historique de mon code à chaque étape clé, de sécuriser mes modifications et de travailler efficacement sur différentes branches de développement. Ils font partie de la méthodologie de travail de la *dev-team* (cf. [section 3.4](#)).

3.4 Méthodologie de travail

La méthodologie de travail au sein de la *dev-team* s'inspire des méthodes **Agile**. Ce fonctionnement repose sur des cycles courts, cela offre une grande réactivité face aux imprévus techniques. Chaque semaine, nous tenions des réunions de suivi de projet durant lesquelles chaque membre de l'équipe présentait l'avancement de ses missions, les difficultés rencontrées ainsi que ses éventuelles questions. À l'issue de ces échanges, chacun savait précisément quelles tâches accomplir pour la semaine à venir.

Pour la communication au sein de la *dev-team*, nous utilisons **Mattermost**. C'est une plateforme de messagerie instantanée. Ainsi, dès que je devais poser une question à mon tuteur, je lui envoyais un message via cet outil. Cela permet d'avoir une communication plus fluide et moins rigide que par mail, même si les mails étaient toujours utilisés, notamment pour nous informer des horaires de réunion.

De plus **Mattermost** inclut des « *boards* ». C'est un tableau de tâches en ligne visible par tous. Pour chaque tâche effectuée, on indique une description courte, l'état de finition, le

degré d'urgence et à qui elle a été assignée. Cet outil m'a permis de partager l'avancée de mes missions avec les autres membres de l'équipe de manière transparente.

Pour ce qui est du *workflow*, nous utilisons **Gitflow** et **Gitlab**. **Gitflow** est un modèle de collaboration qui utilise **git**. Ici, il n'y a que deux branches permanentes, la branche **main** et la branche **develop**. La branche **main** est exclusivement dédiée à la production, ainsi seule une version testée et fonctionnelle de MORFETIK peut y être présente. La branche **develop** sert quant à elle d'intégration pour les fonctionnalités et constitue la base de travail réelle pendant le développement.

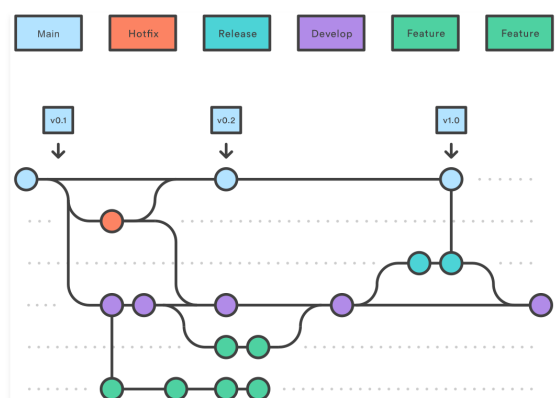


FIGURE 3.4 – Schema de la gestion des versions avec Gitflow

De plus, **Gitflow** utilise des préfixes dans le nom des branches pour savoir exactement quelle branche apporte quoi. Par exemple, lorsque je voulais ajouter une nouvelle fonctionnalité à MORFETIK, j'utilisais le préfixe **feature/** suivi du nom de ma branche.

Exemple de titre : `feature/migration_mongodb_to_postgresql`

Une fois une branche créée, on peut travailler dessus. Chaque commit suit la norme des **conventional-commits** afin d'avoir un historique clair. Cette norme impose d'ajouter un préfixe avant notre message de commit. Ce message se doit d'être explicite et court afin de décrire au mieux le changement effectué. Donc, si le commit concerne l'implémentation d'une fonctionnalité, on se doit d'utiliser le préfixe **feat** ou alors, si l'on veut ajouter une documentation, on peut utiliser **docs**. Il existe une multitude de préfixes pour décrire tous les changements possibles (cf. **Annexe 2**).

Une fois mon travail terminé sur ma branche, je dois effectuer une **Merge Request (MR)**, autrement dit une « demande de fusion ». Cette étape consiste à soumettre mes modifications à mon tuteur de stage, afin qu'il effectue une revue de mon travail. Cela permet de garantir que mes modifications soient pertinentes pour MORFETIK et respectent les critères de qualité exigés, tels que la propreté du code et la documentation.

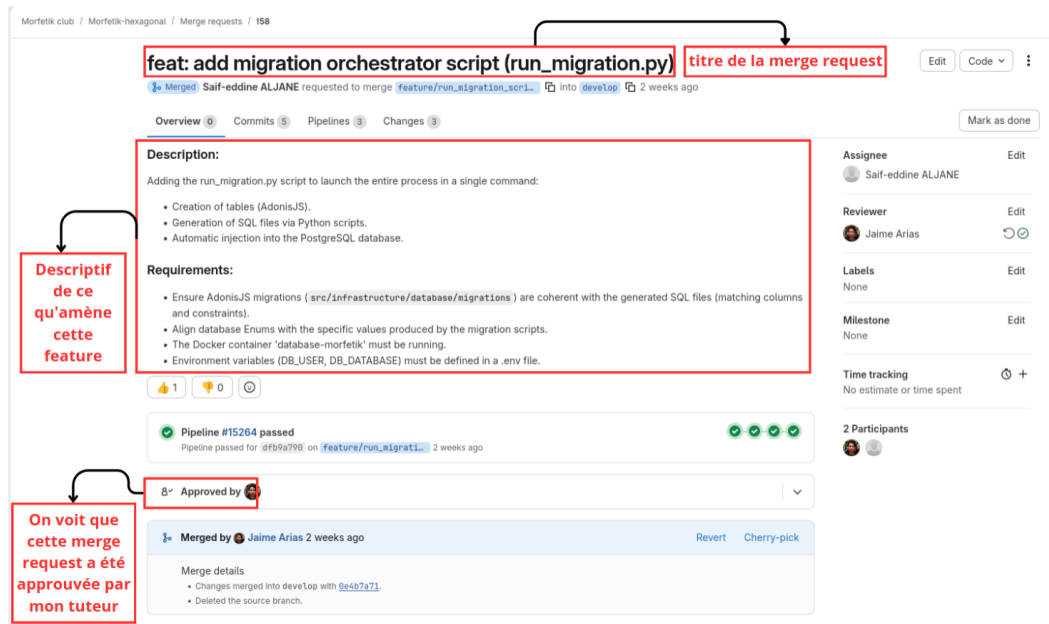


FIGURE 3.5 – Revue de code lors d’une demande de fusion (merge request)

Ainsi, l’utilisation de la méthode Agile, de Gitflow et des Conventional Commits permet de faire en sorte que MORFETIK (la nouvelle version) soit maintenable sur le long terme car l’historique de développement est clair et précis.

Analyse et réalisations techniques

Avant d’entamer le développement de la solution de migration, j’ai dû me familiariser avec le code et le travail déjà existant. Pour récapituler, lors de mon arrivée, il y avait l’ancienne version de MORFETIK d’un côté, et de l’autre *Morfetik-hexagonal* (le nom de la nouvelle version). Cette nouvelle version comprenait déjà le **back-end** de l’application et une version de la base de données POSTGRESQL. La base POSTGRESQL existait déjà et mon rôle était de migrer les données vers celle-ci. Cependant, grâce aux analyses que j’ai faites pour comprendre l’ancienne version de MORFETIK, j’ai relevé des éléments qui ont fait évoluer la nouvelle base de données en la complétant. J’ai donc dû analyser et comprendre tout le travail qui avait été fait avant moi.

J’ai aussi remarqué que la méthode de travail que l’on m’imposait était très utile. En effet, sur l’ancienne version de MORFETIK, la relecture était plus difficile car les conventions n’étaient pas toujours suivies (cf. [section 3.4](#)), du moins jusqu’à la reprise du projet par mon tuteur. À l’inverse, la nouvelle version *Morfetik-hexagonal* suit entièrement cette méthodologie, ce qui a facilité mon analyse et ma compréhension de la structure cible.

4.1 Analyse des données de Morfetik

J’ai commencé par analyser la base de données POSTGRESQL. Pour ce faire, j’ai utilisé l’**ERD** présent dans la documentation de MORFETIK (cf. [Annexe 3](#)), tout en m’appuyant sur les fichiers de migration d’**AdonisJS** pour comprendre précisément le typage et les contraintes de chaque table :

```

import { BaseSchema } from '@adonisjs/lucid/schema'
// Ce fichier permet à AdonisJS (le back-end) de créer la table noun_codes
export default class extends BaseSchema {
  // Définition du nom de la table dans la base de données PostgreSQL
  protected tableName = 'noun_codes'

  public async up() {
    this.schema.createTable(this.tableName, (table) => {
      // Définition d'un identifiant unique (UUID) avec contrainte de clé primaire
      table.uuid('id').primary()

      // Configuration des colonnes avec contrainte "NOT NULL" pour garantir
      // l'intégrité des données au niveau de PostgreSQL
      table.integer('radical').nullable()
      table.string('model').nullable()
      table.string('singular').nullable()
      table.string('plural').nullable()
    })
  }

  // Méthode de rollback permettant de supprimer la table et ses contraintes
  public async down() {
    this.schema.dropTable(this.tableName)
  }
}

```

FIGURE 4.1 – Définition et paramétrage des contraintes d'intégrité de la table nouns_codes

L'analyse de ces fichiers m'a permis de comprendre que la base de données contient **13 tables**, que l'on peut diviser en quatre catégories logiques :

- **La table « form »** : Elle regroupe toutes les formes de tous les lemmes de la base.
- **Les tables de lemmes (8 tables)** : Ce sont les tables spécifiques à chaque catégorie grammaticale (*i.e.* verbe, adjectif, adverbe, adjectif composé, nom composé, nom, autre, et pronom).
- **Les tables de codes (3 tables)** : Elles servent de référence pour les lemmes et contiennent les propriétés grammaticales ainsi que les modèles de flexion.
- **La table « domain »** : Cette treizième table contient les noms des domaines associés aux mots (*e.g.* agriculture, informatique, etc.). À l'origine, les domaines n'étaient pas gérés via une table, mais via un *enum* directement dans le code source.

```

/** * Liste originelle des domaines
 */
export const DomainList = [
  '',
  'ADMINISTRATION',
  'AÉRONAUTIQUE',
  'AGRICULTURE',
  'ALIMENTATION',
  /* ... (La liste complète contient plus de 90 domaines) ... */
  'VOYAGES',
  'ZOOLOGIE',
] as const

// Ce type définit une règle de validation stricte : il garantit l'intégrité des données
// en limitant les choix possibles aux seules valeurs présentes dans le tableau.
export type Domain = (typeof DomainList)[number]

export default Domain

```

FIGURE 4.2 – Dictionnaire des domaines

Le choix de passer par une table dédiée dans la nouvelle version permet de rendre la gestion des domaines beaucoup plus souple. Concernant les relations, la table `form` est liée à chaque table de lemme via une **clé étrangère** (*Foreign Key*). Cette clé est un **UUID** ; utiliser ce type d'identifiant est bien plus robuste que d'utiliser un simple entier auto-incrémenté, car il garantit l'unicité même en manipulant de gros volumes de données.

De la même manière, les tables `verb`, `adjective` et `noun` sont liées aux tables de codes correspondantes. Ce lien permet de récupérer les modèles de conjugaisons ou les flexions de manière dynamique. Enfin, chaque table possédant un domaine est liée à la table `domain` via une clé étrangère.

Pour conclure, l'analyse de cette base de données ne m'a pas posé de problème particulier, du fait que dans mon cursus j'ai déjà été amené à travailler avec des bases de données relationnelles. J'ai donc rapidement compris l'organisation des données au sein de *Morfetik-hexagonal*, ce qui m'a laissé plus de temps pour bien appréhender la structure plus complexe de la version MONGODB.

L'analyse de la base de données MONGODB a été plus complexe que celle de la version PostgreSQL, car c'était la première fois que j'étais confronté à ce type de base de données non relationnelle (NoSQL). Pour mener à bien cette analyse, je me suis appuyé sur deux supports principaux :

- **Les fichiers JavaScript** : Ils faisaient office de schémas pour les documents de chaque collection, ce qui m'a permis de comprendre la structure théorique des données (cf. [Annexe 4](#)).
- **Le Shell MONGODB** : Je l'ai utilisé pour effectuer des requêtes directement sur la base afin de visualiser et de vérifier les données réelles.

Sur ce point, l'outil **Docker** m'a été d'une grande aide. Il m'a permis de lancer instantanément un environnement MONGODB local sans avoir à m'occuper de l'installation et de la configuration du service sur ma machine. Ce gain de temps a été précieux pour me permettre de me concentrer rapidement sur l'analyse des données.

Pour interagir avec la base contenue dans le conteneur, l'accès se fait via une commande simple permettant d'exécuter un terminal interactif à l'intérieur du conteneur Docker :

```
docker exec -it morfetik-database mongo
```

L'analyse des données m'a demandé de me familiariser avec la syntaxe des requêtes MONGODB qui, contrairement au SQL, suit une notation objet proche du JavaScript. La structure d'une commande respecte toujours la hiérarchie suivante :

1. **db** : l'objet représentant la base de données active.
2. **La collection** : l'espace où sont stockés les documents (*e.g.* verbs, nouns).
3. **La méthode** : l'action à réaliser (recherche, comptage, mise à jour).

Par exemple, la requête `db.verbs.findOne()` retourne un unique document de la collection "verbs", ce qui m'a été très utile pour visualiser les données concrètes.

Pour ce qui est de la structure, elle suit une logique similaire à celle de la base PostgreSQL : les collections ont été triées en plusieurs catégories. Cependant, le modèle

MONGODB introduit une granularité supplémentaire avec des catégories et sous-catégories :

- **Les collections de lemmes** : associées à chaque classe grammaticale.
- **Les collections de codes** : elles regroupent les propriétés grammaticales ainsi que les modèles de flexion (*cf.* [Annexe 5](#)).
- **La collection domaine** : pour définir le contexte thématique.
- **La collection formes** : elle contient toutes les flexions (les mots accordés) liées aux lemmes.

La collection **formes** constitue le **pivot de la base de données**. Elle assure le lien entre les lemmes et les flexions. On constate qu'elle contient l'ObjectID du lemme associé, ce qui correspond au concept de **clé étrangère** en SQL.

La principale difficulté rencontrée a été de visualiser les objets imbriqués, particulièrement pour la collection `verbscodes`. Comme chaque document doit contenir l'ensemble des terminaisons pour chaque temps et chaque mode, la structure devient rapidement conséquente.

Pour modéliser ces données au format JSON, une structure imbriquée sur **cinq niveaux** est devenue indispensable :

- **Niveau 1** : Les modes (*e.g.* indicatif, subjonctif, impératif, etc.).
- **Niveau 2** : Les temps (*e.g.* présent, imparfait, futur, etc.).
- **Niveau 3** : Le nombre (*i.e.* singulier ou pluriel).
- **Niveau 4** : La personne (*i.e.* 1ère, 2ème, 3ème) ou le genre.
- **Niveau 5** : La valeur finale (la terminaison).

Ainsi, pour accéder à la terminaison d'un verbe à la première personne du présent de l'indicatif, le chemin d'accès est le suivant : `indicatif.present.singulier.1`.

4.2 Conception et développement de la solution de migration

Ma solution de migration repose sur le développement de scripts écrits en **Python**. Ces scripts ont pour rôle d'extraire les données de la base de données source, de les transformer, puis de générer un fichier de sortie adapté aux standards de la base cible.

Cette démarche s'inscrit dans une logique de flux de données appelée **ETL** (*Extract, Transform, Load*) :

- **Extraction (Extract)** : Récupération des données brutes depuis la base de données source (MONGODB).
- **Transformation (Transform)** : Pour passer du format **NoSQL** au format SQL, le script doit adapter les documents MONGODB aux tables SQL en les transformant en requête "INSERT INTO".
- **Chargement (Load)** : Génération d'un fichier SQL final permettant l'injection des données transformées dans la base de destination.

L'utilisation de cette méthode ETL est indispensable car elle permet de garantir l'**intégrité des données** et d'assurer qu'aucune information n'est perdue ou corrompue lors du changement de format.

Pour réaliser cette migration, j'ai conçu une solution composée de **14 scripts Python**. Cette organisation permet de séparer les responsabilités et de faciliter la maintenance du code :

- **13 scripts de transformation** : Chaque script puise les données dans une collection MONGODB et génère un fichier SQL qui contient les données d'une table SQL.
- **1 script d'orchestration** : Ce script pilote l'ensemble du processus. Il gère l'ordre d'exécution des 13 autres scripts et s'occupe du chargement final des données dans la base POSTGRESQL.

Voici un schéma qui permet de mieux visualiser ce que font les 13 scripts de transformation.

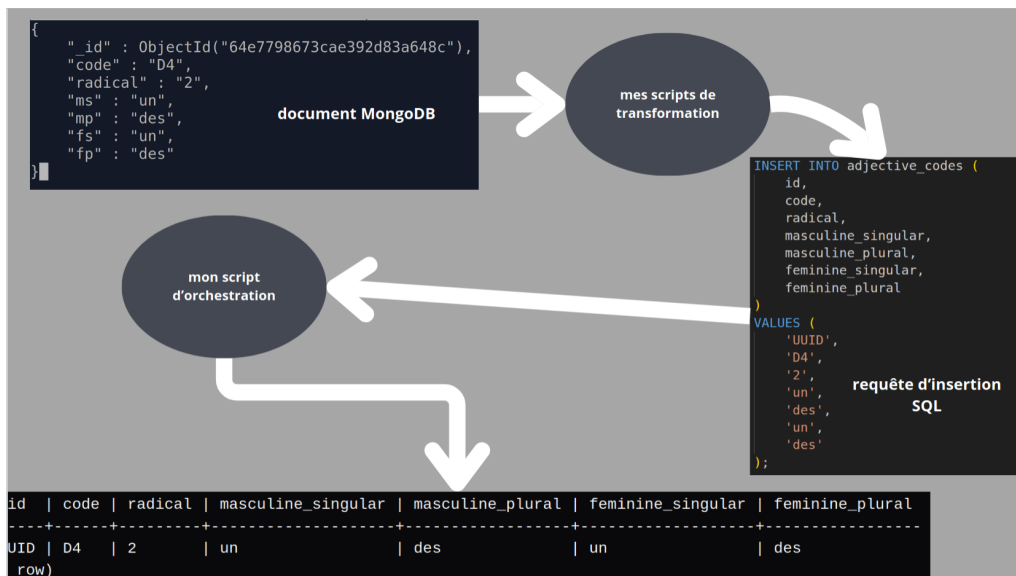


FIGURE 4.3 – Processus de transformation et d'insertion des données

L'interaction avec la base de données source repose sur la bibliothèque Pymongo. Cette bibliothèque permet de charger chaque collection dans une variable sous forme de **liste de dictionnaires**. Comme un document MONGODB possède une structure similaire à un dictionnaire Python, cette méthode permet de manipuler les données de manière native.

Tous mes scripts de transformation reposent sur un même algorithme qui s'adapte selon le contexte de la collection. Il consiste à récupérer les données des documents pour les convertir en requêtes SQL valides. Ce processus se décompose en cinq étapes clés :

- **Récupération des données** : Via la bibliothèque pymongo, la collection est chargée en mémoire. Chaque document est traité comme un dictionnaire Python, ce qui permet d'accéder aux données par leurs clés de manière native.
- **Parcours des documents** : Le script boucle sur chaque document de la collection

chargée. C'est ici que l'algorithme s'adapte : selon la collection, les documents peuvent avoir une structure simple ou, au contraire, être très complexes à cause de l'imbrication des données.

- **Aplatissement et mapping des données** : Pour les documents complexes (comme dans `verbscodes`), l'algorithme "aplatit" la structure. Il descend dans les niveaux d'imbrication pour extraire la valeur finale.
- **Création de la requête** : Le script prend les données récupérées et construit une requête `INSERT INTO`. C'est aussi à ce moment que l'on valide les données pour voir si elles correspondent aux contraintes de la table. Ainsi pour chaque document il y aura une ligne dans la table SQL.
- **Génération du fichier SQL** : Enfin, le script écrit chaque requête dans un fichier SQL dédié. Ces fichiers sont stockés dans un répertoire `sql_file` qui est généré dynamiquement par le script au début de l'exécution.

Pour automatiser le processus, j'ai conçu un script d'orchestration qui coordonne treize scripts de migration selon un ordre de priorité strict, garantissant ainsi l'intégrité des clés étrangères.

Le processus se déroule en trois étapes :

- **Initialisation du schéma** : J'utilise ici le script `init-db.js` réalisé par un collaborateur. Ce script prépare l'environnement en créant la base de données et en lançant les migrations AdonisJS pour générer la structure des tables.
- **Génération ordonnée** : Exécution séquentielle des scripts Python pour transformer les données MONGODB et générer les fichiers SQL correspondants.
- **Injection Docker** : Automatisation de l'import via `psql` dans le conteneur, incluant une surveillance des erreurs en temps réel pour stopper le processus en cas d'anomalie.

Sachant que mes scripts de transformation se ressemblent tous en globalité vous aurez en annexe seulement un de mes scripts de transformation et mon script d'orchestration ([Annexe 6](#) et cf. [Annexe 7](#))

4.3 Analyse des problématiques techniques et solutions apportées

4.3.1 Gestion des clés étrangère

La première difficulté que j'ai rencontrée concerne les tables qui possèdent des clés étrangères. En effet, pour les tables de codes qui sont liées aux tables `verbs`, `adjective` et `noun`, le lien dans MONGODB était une simple chaîne de caractères (par exemple "01"). Le problème est qu'il n'est pas possible de garder ce lien tel quel dans la version PostgreSQL : les clés étrangères doivent impérativement être des **UUID**.

Ainsi, pour conserver les liens entre les données, j'ai fait en sorte que pour chaque document des collections concernées, mon script génère un **tuple** composé du code (chaîne de caractères) et de l'UUID généré. Ces tuples sont ensuite ajoutés dans une liste Python.

```
# Extraction du code source (ex: "01")
mongo_code = doc.get("code")

# Génération de l'identifiant unique (ex: 069aecad-eaf5-70e4-8000-4f53744ab998)
uuid = uuid7()

# Création du tuple de correspondance (Mapping)
# Format : (Code MongoDB, UUID PostgreSQL)
mapping_tuple = (mongo_code, uuid)

# Ajout à la liste pour traitement futur par la fonction get_uuid
code_to_uuid_list.append(mapping_tuple)
```

FIGURE 4.4 – Création de la liste de tuples de correspondance Code/UUIDv7.

En passant cette liste de tuples en paramètre de ma fonction `get_uuid`, celle-ci génère un fichier Python contenant un **dictionnaire**. Ce dernier utilise les codes comme clés et les UUID correspondants comme valeurs.

Cela rend le lien **persistant** et parfaitement adapté aux contraintes de PostgreSQL. Il ne reste plus qu'à importer ce dictionnaire dans les scripts suivants pour récupérer le bon UUID simplement en utilisant le code d'origine comme clé.

4.3.2 Les documents imbriqués

J'ai aussi eu des difficultés avec la migration de la collection `verbscodes`. Comme la table PostgreSQL `verbs_code` contient l'intégralité des terminaisons de conjugaison (soit environ 50 colonnes), l'extraction manuelle via la méthode `.get()` n'est pas viable. De plus, pour récupérer une terminaison, il faut parfois parcourir 5 niveaux d'imbrication.

Ainsi, j'ai créé une fonction (`get_conjugaison_value`) qui parcourt automatiquement ces niveaux en utilisant un dictionnaire de chemins que j'ai nommé `conjugation_path` (cf. [Annexe 8](#)).

Dans ce dictionnaire, les clés correspondent aux noms des colonnes SQL et les valeurs représentent les chemins d'accès dans MONGODB (par exemple : `['indicatif', 'present', 'singulier', '1']`). Ma fonction `get_conjugaison_value` prend ce dictionnaire en paramètre et parcourt l'objet étape par étape :

- **Si elle trouve la donnée** : elle récupère la terminaison correspondante.
- **Si une clé est manquante** : elle renvoie NULL, ce qui sécurise le script et évite qu'il ne s'arrête si une forme de conjugaison est absente.

Ainsi, cette fonction retourne une liste de chaînes de caractères où chaque élément représente une terminaison adaptée au format SQL. Cette approche permet de transformer une structure imbriquée en une ligne de données "plate", prête à être insérée dans PostgreSQL.

4.3.3 Validation et transformation des types

La dernière difficulté que j'ai rencontrée concerne la validation des données. Il était impératif de vérifier que chaque valeur extraite respecte les contraintes de type des colonnes PostgreSQL. Pour automatiser ce nettoyage, j'ai créé trois fonctions : `to_int`, `to_str` et `valide_mapping`.

- `to_int` et `to_str` : Elles permettent de convertir la donnée et de définir une valeur par défaut (NULL) si la source est inexistante, évitant ainsi des erreurs d'insertion.
- `valide_mapping` : Cette fonction valide la donnée par rapport à un dictionnaire de référence et retourne la valeur normalisée (e.g. transformer 'adj' en 'ADJECTIVE').

Ces fonctions m'ont été d'une grande utilité car la base MONGODB contenait des données incohérentes. Ces anomalies sont dues au fait que les données ont subi plusieurs migrations par le passé, ce qui a fini par créer des incohérences.

L'implémentation détaillée des fonctions `get_conjugaison_value`, `to_int`, `to_str` et `valide_mapping` est consultable dans l'annexe dédiée aux fonctions utilitaires (cf. [Annexe 9](#)).

Conclusion et perspectives

Je suis satisfait de l'expérience que j'ai eue au **LIPN**, car j'ai pu voir la réalité du milieu de l'informatique. Avant mon stage, j'appréciais le domaine d'un point de vue académique, mais je me demandais si ça me plairait autant dans un contexte professionnel. Ce stage m'a confirmé que c'est un milieu qui me convient vraiment.

Je suis aussi satisfait du travail que j'ai fait pour la migration de MORFETIK. Même s'il faut attendre que le projet Morfetik-hexagonal avance encore pour lancer la migration finale, je sais que mes scripts seront utilisés pour transférer les données. Comme j'ai bien documenté mon travail, l'équipe pourra l'adapter facilement si le contexte l'exige.

Pour réussir la migration de MORFETIK, j'ai dû mobiliser plusieurs compétences de ma formation. Comme mon travail s'est concentré sur la gestion des données (Compétence 4¹), cela m'a permis de comprendre MONGODB et de réussir à adapter deux formats de données très différents, du NoSQL vers le SQL.

Les compétences 1² et 2³ m'ont permis de concevoir mes scripts de migration de manière structurée (cf. **Annexe 10**). Enfin, la compétence 6⁴ m'a permis de plus facilement m'imprégner de la méthode de travail de l'équipe, notamment en suivant les rituels agiles et les outils de communication de la dev-team (cf. **section 3.4**).

D'un point de vue purement technique, j'ai appris à utiliser MONGODB, et dans une moindre mesure AdonisJS (Node.js) et TypeScript. J'ai aussi appris à utiliser Git de façon professionnelle, notamment avec Gitflow et les "Conventional Commits", car auparavant je ne l'utilisais pas de la bonne façon. J'ai aussi appris à utiliser Docker, qui permet d'harmoniser l'environnement de développement et de garantir que le projet fonctionne de la même manière sur n'importe quelle machine. De plus, je devais aussi travailler sur l'interface utilisateur de la nouvelle version de MORFETIK, mais par manque de temps, cela ne s'est pas fait.

1. Compétence 4 : Gérer des données de l'information
2. Compétence 2 : Optimiser des applications informatiques
3. Compétence 1 : Réaliser un développement d'application
4. Compétence 6 : Travailler dans une équipe informatique

Et d'un point de vue humain, j'ai découvert comment travailler au sein d'une équipe de développement, mais aussi comment expliquer son travail et poser les bonnes questions quand on a des difficultés.

Côté perspectives, je tiens à poursuivre mes études après l'obtention de mon BUT. Cette formation m'a apporté des bases techniques solides, et je souhaite désormais intégrer une école d'ingénieur pour me spécialiser.

Bibliographie

- [1] LIPN. *Documentation du projet MORFETIK*. 2026.
- [2] LIPN. *Site officiel du laboratoire*. 2026. URL : <https://lipn.univ-paris13.fr/>.
- [3] CNRS. *Site institutionnel du CNRS*. 2026. URL : <https://www.cnrs.fr/>.
- [4] MONGODB. *Manuel officiel (NoSQL)*. 2026. URL : <https://www.mongodb.com/docs/>.
- [5] ADONISJS. *Documentation du framework (Node.js)*. 2026. URL : <https://docs.adonisjs.com/>.

Glossaire

A

ACID

Ensemble de quatre règles (Atomicité, Cohérence, Isolation, Durabilité) qui garantissent qu'une opération sur la base est fiable et qu'aucune donnée ne sera perdue ou corrompue. p. 16

Agile

Méthode de gestion de projet basée sur des cycles de travail courts. Elle permet d'être très réactif face aux imprévus techniques rencontrés par l'équipe . p. 17, 19

B

Back-end

Désigne la partie "invisible" d'une application. Elle regroupe la logique métier, la gestion de la base de données et la configuration du serveur . p. 16, 20

BSON

Format de stockage utilisé par MongoDB pour transformer les objets JSON en données binaires. Cela permet de gagner de la place et de mieux gérer des données comme les dates. p. 15

C

CNRS

Centre National de la Recherche Scientifique. p. 8, 9, 11, 14

Conventional Commits

Convention de nommage pour les messages de commit. Elle impose des préfixes comme "feat" ou "fix" pour rendre l'historique de développement lisible . p. 19

E

EPST

Établissements Publics à caractère Scientifique et Technologique. p. 8

ERD

Schéma graphique modélisant les entités et les relations d'une base de données. Indispensable pour la conception relationnelle . p. 20

F

Flexion

Désigne les modifications subies par un mot (lemme) pour s'adapter au contexte grammatical (genre, nombre, temps). Pour un verbe, l'ensemble de ces variations forme ce qu'on appelle la conjugaison. p. 13, 14

G**Gitflow**

Modèle d'organisation des branches Git. Il définit comment séparer le travail en cours (develop) de la version stable et fonctionnelle (main) . p. 18, 19

I**IDE**

Integrated Development Environment. p. 17

J**JSON**

Format de données qui représente les informations sous forme d'objets (clés et valeurs). Sa structure est simple à lire et très proche des dictionnaires utilisés en Python. p. 15

L**L2TI**

Laboratoire de Traitement et Transmission de l'Information. p. 9

LAGA

Laboratoire d'Analyse, Géométrie et Application. p. 9

Lemme

Forme de référence d'un mot, (par exemple, l'infinitif pour un verbe ou le masculin singulier pour un adjectif). p. 13, 14

LIPN

Laboratoire d'Informatique de Paris Nord. p. 2, 5, 6, 8–11, 13, 28

M**MESRI**

Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation. p. 8

MR

Action de soumettre son code à une vérification par un tuteur ou un collègue avant de l'intégrer au projet principal. C'est une étape de contrôle qualité. p. 18

N**Node.js**

Environnement d'exécution qui permet d'utiliser le langage JavaScript pour développer des applications côté serveur. Il sert de base au framework AdonisJS. p. 16

NoSQL

Modèle de base de données s'affranchissant des structures relationnelles classiques afin de privilégier la flexibilité de stockage et la gestion de données non structurées. p. 14, 15, 23

O**ORM**

Permet de faire le lien entre les objets du code source (en TypeScript/Python) et les tables d'une base de données relationnelle (PostgreSQL). Lucid est l'ORM utilisé par AdonisJS. p. 16

S**SGBD**

Logiciel qui permet de gérer une base de données (stockage, sécurité, accès). C'est l'outil qui fait l'interface entre l'utilisateur et les données. p. 15, 16

SGBDR

Type de base de données où les informations sont rangées dans des tables liées entre elles. C'est le modèle utilisé par PostgreSQL pour garantir l'ordre des données. p. 15

SPA

Single Page Application. p. 17

T**TAL**

Disipline qui a pour objectif de permettre à des systèmes informatiques de comprendre, d'analyser et de générer du langage humain (écrit ou oral) de manière autonome. Dans notre contexte, Morfetik est une ressource qui peut servir à étudier la structure des mots et leurs déclinaisons.. p. 13

U**UMR**

Unité mixte de recherche. p. 8

USPN

Université Sorbonne Paris Nord. p. 8, 9

UUID

Identifiant unique de 128 bits assurant l'unicité globale des données et évitant les collisions lors de migrations complexes . p. 22

Annexes

Annexe 1 : Statistiques de la base de données MORFETIK

Statistiques Morfetik		
Total des graphies		Total des valeurs (compte tenu des homographies)
248645		1031833
Type de graphie	Nombre de graphies	Nombre de valeurs
Noms simples	70058	138795
Noms composés	131982	265800
Verbes	10230	521730
Adjectifs simples	24407	62838
Adjectifs composés	9524	40228
Type de graphie	Nombre de graphies/valeurs	
Adverbes	1899	
Pronoms	123	
Déterminants	141	
Autres graphies	281	

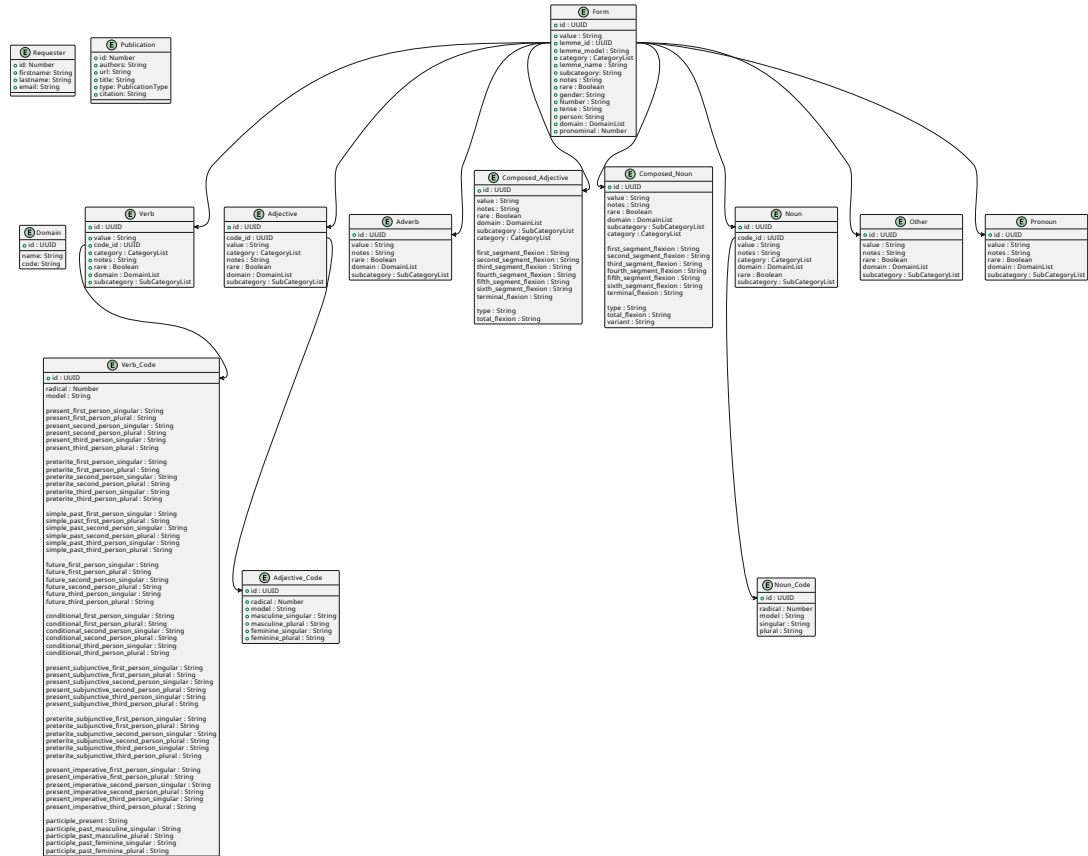
Source : Statistiques extraites du site officiel de MORFETIK.

Annexe 2 : Préfixes pour les commits

Conventional Commits	
How to write good commits	
Type	Meaning
feat	A new feature
fix	A bug fix
docs	Documentation changes only
style	Formatting, missing semi-colons, etc.
refactor	Code change that neither fixes nor adds
test	Adding/correcting tests
build	Build system or external dependencies
ci	CI configuration files and scripts
chore	General maintenance tasks

Source : Extrait du support de présentation, réalisé par M. ARIAS.

Annexe 3 : Schéma relationnel de la base de données PostgreSQL (ERD)



Source : Documentation du projet MORFETIK, schéma réalisé par Othman BOUDARGA.

Note : N'hésitez pas à utiliser le zoom pour mieux visualiser le schéma.

Annexe 4 : Modèle de données de la collection « Nouns »

Source : fichiers *Nouns.js* provenant de l'ancienne version de MORFETIK

```
1 const mongoose = require('mongoose');
2 const mongoosePaginate = require('mongoose-paginate-v2');
3 const Schema = mongoose.Schema;
4
5 // Structure que chaque document de la collection "Nouns" va avoir
6 const SpecificSchema = new Schema({
7   name: {
8     type: String,
9     required: true // Rend la présence du champ "name" obligatoire
10  },
11  catgram: { type: String, required: true },
12  souscat: String, // Ces champs ne sont pas obligatoires (MongoDB schema-less)
13  flexion: [String],
14  domaine: String,
15  variante: String,
16  infos: String,
17  notes: String
18 });
19
20 SpecificSchema.plugin(mongoosePaginate);
21 // Compilation du modèle
22 const Model = mongoose.model('Nouns', SpecificSchema);
23 module.exports = Model;
```

Annexe 5 : Exemple d'un document de la collection « Nouns »

Tous les noms possédant le code « 00 » ne changent pas de forme, que ce soit au singulier ou au pluriel.

```
1 {
2   "_id": "64e7798673cae392d83a6497",
3   "code": "00",
4   "radical": "0",
5   "s": "",
6   "p": ""
7 }
```

Annexe 6 : Exemple d'un script de migration de données

```

1 import os
2 from uuid_extensions import uuid7
3 from migration_helpers.enum.subcategoryList import subcategoryList
4 from migration_helpers.tools import to_str, valide_mapping, get_uuid
5 from mongoDB_connection import db
6
7 try:
8     from migration_helpers.uuid_mappings.domains_mapping import code_to_uuid_for_domains
9 except ModuleNotFoundError as e:
10     print(f"Erreur : Mapping manquant ({e.name})"); exit(1)
11
12 # --- Configuration du fichier SQL ---
13 directory, file_name = "sql_file", "migrate_adverbs_data.sql"
14 if not os.path.exists(directory): os.makedirs(directory)
15 path = os.path.join(directory, file_name)
16
17 # --- Traitement des données ---
18 with open(path, "w", encoding="utf-8") as f:
19     adverbs_collection = db.adverbs.find()
20     all_values_tab, objectID_to_uuid = [], []
21
22     for adverbs_doc in adverbs_collection:
23         adverbs_id = adverbs_doc.get("_id", "")
24         # Récupération des données associées dans la collection 'formes'
25         formes_doc = db.formes.find({"lemme._id": adverbs_id, {"rare": 1, "domaine": 1, "_id": 0}})
26         for doc in formes_doc:
27             mongo_rare = bool(doc.get("rare", ""))
28             dom = doc.get("domaine", "")
29             uuid_domain = code_to_uuid_for_domains.get(dom, code_to_uuid_for_domains["NULL"])
30
31             # Identifiants et normalisation pour SQL
32             uuid = uuid7()
33             objectID_to_uuid.append((adverbs_id, uuid))
34             mongo_name = to_str(adverbs_doc.get("name", ""), "NULL")
35             mongo_note = to_str(adverbs_doc.get("notes", ""), "NULL")
36             mongo_souscat = valide_mapping(adverbs_doc.get("souscat", ""), subcategoryList)
37
38             # Création de la ligne d'insertion
39             val = f"({'{uuid}', {mongo_name}, {mongo_note}, {mongo_rare}, '{uuid_domain}',
40 ↪      {mongo_souscat}})"
41             all_values_tab.append(val)
42
43 # --- Génération du fichier SQL final ---
44 insert_query = "INSERT INTO adverbs (id, value, notes, rare, domain, subcategory) VALUES\n" +
45 ↪      ",\n".join(all_values_tab) + ";"
46 get_uuid(objectID_to_uuid, "adverbs_mapping.py", "objectID_to_uuid_for_adverbs")
47 f.write("-- Données générées par le script de migration\n" + insert_query)
48 print(f"Succès : Fichier SQL généré à l'emplacement {path}")

```

Annexe 7 : Script d'orchestration

```

1 import os, subprocess
2 from dotenv import load_dotenv
3 from pathlib import Path
4
5 # Chargement des variables d'environnement (.env)
6 load_dotenv()
7 user, db = os.getenv("DB_USER"), os.getenv("DB_DATABASE")
8 container = "database-morfetik"
9
10 # Définition des chemins (compatibilité multi-plateforme)
11 scripts_dir = Path(__file__).parent / "migration"
12 sql_dir = Path(__file__).parent / "sql_file"
13
14 # Ordre d'exécution des scripts de génération des fichiers sql
15 scripts_order = [
16     "migrate_noun_code.py", "migrate_adjective_code.py",
17     # [...] Liste abrégée pour le rapport (13 fichiers)
18     "migrate_adverb.py", "migrate_other.py", "migrate_formes.py"
19 ]
20
21 # Ordre d'exécution des fichiers SQL
22 sql_order = [
23     "migrate_noun_code_data.sql", "migrate_adjective_code_data.sql",
24     # [...] Liste abrégée correspondante
25     "migrate_adverbs_data.sql", "migrate_other_data.sql", "migrate_forms_data.sql"
26 ]
27
28 def init_db():
29     # Initialisation de la base de données et des migrations AdonisJs
30     try:
31         subprocess.run(["npm", "run", "db:init"], check=True)
32         subprocess.run(["npm", "run", "db:migrate"], check=True)
33     except subprocess.CalledProcessError as e:
34         print(f"Erreur init DB : {e}"); exit(1)
35
36 def create_sql_file(order, directory):
37     # Lancement des scripts de génération SQL
38     for script in order:
39         subprocess.run(["python", directory / script], check=True)
40
41 def execute_sql(order, directory, container, user, db):
42     # Injection SQL dans le conteneur Docker "database-morfetik"
43     cmd = ["docker", "exec", "-i", container, "psql", "-U", user, "-d", db]
44     for sql_file in order:
45         with open(directory / sql_file, "r") as f:
46             res = subprocess.run(cmd, stdin=f, capture_output=True, text=True)
47             if "ERROR" in res.stderr or "ERROR" in res.stdout:
48                 print(f"Erreur SQL ({sql_file}) : {res.stderr}"); exit(1)
49
50 if __name__ == "__main__":
51     if not all([user, db]):
52         print("Erreur : Variables DB_USER ou DB_DATABASE manquantes"); exit(1)
53
54     print("--- DÉMARRAGE MIGRATION ---")
55     init_db() # 1. Init schéma
56     create_sql_file(scripts_order, scripts_dir) # 2. Générer SQL
57     execute_sql(sql_order, sql_dir, container, user, db) # 3. Injecter SQL
58     print("MIGRATION TERMINÉE")

```

Annexe 8 : Dictionnaire des chemins de conjugaison (conjugation_path)

```

1  # Clé : Nom de la colonne SQL | Valeur : Chemin d'accès ordonné dans MongoDB
2  conjugation_path = {
3      # --- INDICATIF ---
4      # Présent
5      "present_first_person_singular": ["indicatif", "present", "singulier", "1"],
6      "present_first_person_plural": ["indicatif", "present", "pluriel", "1"],
7      # [...] Suite des personnes pour le Présent, l'Imparfait, le Passé Simple et le Futur
8
9      # --- CONDITIONNEL ---
10     "conditional_first_person_singular": ["conditionnel", "present", "singulier", "1"],
11     # [...] Suite du Conditionnel Présent
12
13     # --- SUBJONCTIF ---
14     "present_subjunctive_first_person_singular": ["subjunctif", "present", "singulier", "1"],
15     "present_subjunctive_first_person_plural": ["subjunctif", "present", "pluriel", "1"],
16     # [...] Suite du Subjonctif Présent et Imparfait
17
18     # --- IMPÉRATIF ---
19     "present_imperative_first_person_singular": [], # Forme inexistante
20     "present_imperative_first_person_plural": ["imperatif", "present", "pluriel", "1"],
21     "present_imperative_second_person_singular": ["imperatif", "present", "singulier", "2"],
22     # [...] Suite de l'Impératif
23
24     # --- PARTICIPE ---
25     "participle_present": ["participe", "present"],
26     "participle_past_masculine_singular": ["participe", "passe", "singulier", "m"],
27     "participle_past_masculine_plural": ["participe", "passe", "pluriel", "m"],
28     # [...] Suite des accords (féminin, pluriel)
29 }

```

Annexe 9 : Fonctions utilitaires (tools.py)

```

1  import json, os
2
3  def get_conjugaison_value(doc, conjugaison_path):
4      # Extraction des terminaisons via parcours du dictionnaire MongoDB
5      found, res = True, []
6      for path in conjugaison_path.values():
7          verb_ending = doc
8          if path == []:
9              res.append("NULL"); continue
10         for step in path:
11             if isinstance(verb_ending, dict) and step in verb_ending:
12                 verb_ending = verb_ending[step]
13             else:
14                 found = False; break
15         res.append(f'"{verb_ending}"' if found else "NULL")
16     return res
17
18  def to_int(v, default):
19      # Conversion int safe
20      if v == "" or v is None: return default
21      try: return int(v)
22      except: return default
23
24  def to_str(v, default):
25      # Conversion string safe (SQL escape)
26      if v == "" or v is None: return default
27      return f'"{str(v).replace("'", "'')}"'
28
29  def valide_mapping(v, mapping):
30      # Validation via dictionnaire de mapping (fallback NULL)

```



```
31     if not v: return "NULL"
32     valide_v = str(v).strip()
33     return f'{{mapping.get(valide_v, 'NULL')}}'
34
35 def get_uuid(code_uuid_list, filename, dict_name):
36     # Génération du fichier de mapping Python (code: uuid)
37     curr_dir = os.path.dirname(os.path.abspath(__file__))
38     out_dir = os.path.join(curr_dir, "uuid_mappings")
39     os.makedirs(out_dir, exist_ok=True)
40
41     mapping = {str(k): str(v) for k, v in code_uuid_list}
42     path = os.path.join(out_dir, filename)
43
44     with open(path, "w", encoding="utf-8") as f:
45         f.write(f'{{dict_name}}= {{json.dumps(mapping, indent=4)}}\n')
46     print(f"Fichier mapping généré : {path}")
```

Annexe 10 : Arborescence du système de migration

```
scripts/
├── run_migration.py           (exécution globale)
├── migration/
│   ├── migrate_[catégorie].py (scripts spécialisés par type grammatical)
│   ├── migration_helpers/     (fonctions utilitaires)
│   ├── uuid_mappings/        (gestion des clés étrangères)
│   ├── mongoDB_connection.py  (connexion sécurisée à la base MongoDB)
└── sql_file/                 (fichiers SQL générés par le script)
```

Structure des répertoires et des fichiers de ma solution de migration.